NYU CS-GY 6033: Design and Analysis of Algorithms I	Spring 2026

Written Homework 3

Name: Riddhi Raina Prasad - rrp4822

Problem 1: Collinear Points

Collaborators: None.

Problem Statement

We are given a set X of n points in $\mathbb{R}^2$ and a query point $\vec{p}$. We need to write a function `max_points_on_line(X, p)` that returns the maximum number of points in X falling on the same straight line containing $\vec{p}$. The algorithm must take **expected time that is linear in the number of points in** X (i.e., $O(n)$ expected time).

Algorithm Description

All points collinear with $\vec{p}$ on a given line share the same *direction* (slope) from $\vec{p}$. Thus, we group points by the slope of the line connecting them to $\vec{p}$ and find the largest group.

Step-by-Step Approach:

1. **Initialize:** Create a hash map `slopes` to count points per slope, a counter `at_p` for points identical to $\vec{p}$, and `max_count` to track the maximum.
2. **Iterate over each point** $(x, y) \in X$:
   - If $(x, y) = \vec{p}$, increment `at_p` and skip (handled separately).
   - Compute the direction vector $(dx, dy) = (x - p_x, y - p_y)$.
   - Reduce (dx, dy) to a canonical form using GCD.
   - Normalize the direction so opposite sides of $\vec{p}$ map to the same slope key.
   - Increment the count for this slope in the hash map and update `max_count`.
3. **Return** `max_count + at_p`.

Edge Cases and Handling

1. **Empty Set:** If $X = \emptyset$, return 0 immediately.
2. **Points Coinciding with $\vec{p}$:** Count separately in `at_p`; add to final result.

3. **Precision:** Use reduced fractions (dx, dy) via GCD instead of floating-point slopes.
4. **Directional Normalization:** Normalize so (dx, dy) and $(-dx, -dy)$ map to the same key (e.g., ensure $dx > 0$, or if $dx = 0$ then $dy > 0$).
5. **Vertical Lines:** $(0, dy)$ normalizes to $(0, 1)$.

Expected Linear Time

The requirement of *expected* linear time means that the algorithm should run in $O(n)$ time in expectation over the distribution of inputs (or equivalently, under the assumption that hash operations behave well on average). Specifically:

- We make exactly one pass over the n points in X, so the loop runs $O(n)$ iterations.
- For each point, we perform: (1) a GCD computation on two integers, which is $O(\log(\max(|dx|, |dy|)))$ in the worst case but $O(1)$ on average for bounded coordinates; (2) a hash table lookup and update. Hash table operations (insert, lookup) have *expected* $O(1)$ time under reasonable hash functions.
- Thus, total expected time is $O(n) \cdot O(1) = O(n)$. We do *not* require worst-case $O(n)$ because hash tables can have worst-case $O(n)$ per operation in adversarial settings; the "expected" qualifier acknowledges this.

Example Walkthrough

Given $X = [(1, 1), (3, 1), (5, 2), (3, 6), (1, 7), (7, 4), (9, 3), (9, 9)]$ and $p = (5, 5)$.

Step 1: For each point, compute $(dx, dy) = (x - 5, y - 5)$, reduce by GCD, and normalize.

- $(1, 1)$: $(-4, -4) \to (-1, -1) \to (1, 1)$. Slope $(1, 1)$.
- $(3, 1)$: $(-2, -4) \to (-1, -2) \to (1, 2)$. Slope $(1, 2)$.
- $(5, 2)$: $(0, -3) \to (0, -1) \to (0, 1)$. Slope $(0, 1)$ (vertical).
- $(3, 6)$: $(-2, 1) \to (-2, 1) \to (2, -1)$. Slope $(2, -1)$.
- $(1, 7)$: $(-4, 2) \to (-2, 1) \to (2, -1)$. Slope $(2, -1)$.
- $(7, 4)$: $(2, -1)$. Slope $(2, -1)$.
- $(9, 3)$: $(4, -2) \to (2, -1)$. Slope $(2, -1)$.
- $(9, 9)$: $(4, 4) \to (1, 1)$. Slope $(1, 1)$.

Step 2: Count points per slope: $(1, 1)$: 2; $(1, 2)$: 1; $(0, 1)$: 1; $(2, -1)$: 4.

Step 3: Maximum count is 4 (slope $(2, -1)$). No points at p. **Return 4.**

The line through $(5, 5)$ with direction $(2, -1)$ (slope $-1/2$) passes through $(1, 7)$, $(3, 6)$, $(5, 2)$, $(7, 4)$ — four points. Correct.

Pseudocode

```
function MAX_POINTS_ON_LINE(X, p)
    if X is empty then
        return 0                                   ▷ Edge case: no points
    end if
    p_x, p_y ← p
    slopes ← empty hash map                        ▷ Count points per slope
    at_p ← 0                                       ▷ Points coinciding with p
    max_count ← 0
    for each (x, y) in X do
        if (x, y) = (p_x, p_y) then
            at_p ← at_p + 1
            continue                               ▷ Handle separately
        end if
        dx ← x − p_x, dy ← y − p_y
        g ← gcd(|dx|, |dy|)                        ▷ Reduce fraction
        slope_x ← dx/g, slope_y ← dy/g
        if slope_x < 0 or (slope_x = 0 and slope_y < 0) then
            Normalize: flip sign of both           ▷ Same line from opposite sides
        end if
        slope ← (slope_x, slope_y)
        slopes[slope] ← slopes[slope] + 1
        max_count ← max(max_count, slopes[slope])
    end for
    return max_count + at_p
end function
```

Python Implementation

```python
import math

def max_points_on_line(X, p):
    if not X:
        return 0  # Edge case: empty set
    p_x, p_y = p
    slopes = {}   # Hash map: slope -> count of points on that line
    at_p = 0      # Count of points exactly at p
    max_count = 0 # Max points on any single line (excluding p)
    for x, y in X:
        if x == p_x and y == p_y:
            at_p += 1
            continue
        dx, dy = x - p_x, y - p_y
        g = math.gcd(abs(dx), abs(dy))  # Reduce to canonical fraction
        slope_x, slope_y = dx // g, dy // g
        if slope_x < 0 or (slope_x == 0 and slope_y < 0):
            slope_x, slope_y = -slope_x, -slope_y  # Normalize direction
        slope = (slope_x, slope_y)
```

```
        slopes[slope] = slopes.get(slope, 0) + 1
        max_count = max(max_count, slopes[slope])
    return max_count + at_p

```

Time Complexity: $O(n)$ expected. **Space Complexity:** $O(n)$.

Problem 2: Adjacency Matrix Properties

Collaborators: None.

Suppose A is the adjacency matrix of a simple undirected graph with n vertices.

Part 1: Squared Matrix A^2

Goal: Show that the (i, j) entry of A^2 is the number of ways to get from i to j in exactly two hops.

Hint (from assignment): Consult your linear algebra notes for the formula for the (i, j) entry of the product of two matrices.

Proof. By the definition of matrix multiplication, the (i, j) entry of $A^2 = A \times A$ is:

$$(A^2)_{ij} = \sum_{k=1}^{n} A_{ik} \cdot A_{kj}$$

For a simple undirected graph, $A_{uv} = 1$ if there is an edge between u and v, and 0 otherwise. Thus $A_{ik} \cdot A_{kj} = 1$ iff both edges (i, k) and (k, j) exist. Each such k contributes exactly one 2-hop path $i \to k \to j$. Summing over k gives the total number of such paths. □

Part 2: Counting Triangles

Goal: Show that $\frac{1}{6}\sum_{i=1}^{n}(A^3)_{ii}$ is the number of triangles in the graph.

Hint (from assignment): Expand the diagonal entries $(A^3)_{ii}$.

Proof. By the formula for matrix multiplication:

$$(A^3)_{ii} = (A \cdot A \cdot A)_{ii} = \sum_{j=1}^{n}\sum_{k=1}^{n} A_{ij}A_{jk}A_{ki}$$

Each nonzero term $A_{ij}A_{jk}A_{ki} = 1$ corresponds to a closed walk of length 3: $i \to j \to k \to i$. Such a walk exists iff i, j, k are pairwise adjacent — i.e., they form a triangle.

Expanding the diagonal: For a triangle $\{i, j, k\}$, the closed walks of length 3 are:

- Start at i: $i \to j \to k \to i$ or $i \to k \to j \to i$ (2 walks)
- Start at j: $j \to i \to k \to j$ or $j \to k \to i \to j$ (2 walks)
- Start at k: $k \to i \to j \to k$ or $k \to j \to i \to k$ (2 walks)

Thus each triangle contributes 6 to $\sum_i (A^3)_{ii}$. Therefore:

$$\text{Number of Triangles} = \frac{1}{6}\sum_{i=1}^{n}(A^3)_{ii}$$

□

Further Reading: GNNs and Triangle Counting

The assignment references the paper by Xu, Hu, Leskovec, and Jegelka: "How powerful are graph neural networks?" (ICLR 2019, https://arxiv.org/abs/1810.00826). This paper shows that standard message-passing Graph Neural Networks (GNNs) have limited expressive power: they cannot reliably distinguish certain graph structures. In particular, GNNs that aggregate node features from neighbors (following a Weisfeiler-Lehman–style scheme) cannot distinguish non-isomorphic graphs that pass the WL test, and they struggle to count substructures like triangles. Triangle counting requires capturing 3-way interactions among nodes, which the local aggregation in GNNs does not inherently capture. The matrix-based formula $\frac{1}{6}\mathrm{tr}(A^3)$ provides an exact $O(n^\omega)$ solution (where ω is the matrix multiplication exponent), whereas GNNs would need to learn such patterns from data — and the theoretical results suggest they may not be able to do so reliably.

Problem 3: Good vs. Evil

Collaborators: None.

Problem Statement

Determine if universities can be labeled "good" or "evil" such that every rivalry (edge) is between a "good" and an "evil" school. If possible, return a dictionary mapping each node to its label; otherwise, return `None`.

Algorithm Description

This is equivalent to checking whether the graph is **bipartite**. We use BFS to attempt a 2-coloring:

1. Iterate over all nodes (to handle disconnected components).
2. For each unvisited node, assign it "good" and run BFS.
3. Assign neighbors the opposite label.
4. If an edge connects two nodes with the same label, return `None`.
5. Otherwise, return the labeling.

Example Walkthrough

Given `example_graph`:

```
{"Michigan": ["Chicago"], "Chicago": ["Michigan","Columbia"],
 "Columbia": ["Chicago","UCSD"], "UCSD": ["Columbia"], "NYU": []}
```

Step 1: Start with Michigan (unvisited). Assign `Michigan = 'good'`, enqueue Michigan.

Step 2: Dequeue Michigan. Neighbors: Chicago. Chicago unvisited $\Rightarrow$ assign `Chicago = 'evil'`, enqueue Chicago.

Step 3: Dequeue Chicago. Neighbors: Michigan (already labeled 'good' $\neq$ 'evil' ✓), Columbia (unvisited $\Rightarrow$ assign `Columbia = 'good'`, enqueue Columbia).

Step 4: Dequeue Columbia. Neighbors: Chicago ('evil' $\neq$ 'good' ✓), UCSD (unvisited $\Rightarrow$ assign `UCSD = 'evil'`, enqueue UCSD).

Step 5: Dequeue UCSD. Neighbors: Columbia ('good' $\neq$ 'evil' ✓), no conflict.

Step 6: NYU is unvisited. Assign `NYU = 'good'` (isolated node; no edges).

Step 7: No conflicts. Return {`"Michigan": "good", "Chicago": "evil", "Columbia": "good", "UCSD": "evil", "NYU": "good"`}.

Edge case (non-bipartite): If we add edge (Michigan, Columbia), then when we process Michigan's neighbors we assign Chicago = 'evil'; when we process Chicago we assign Columbia = 'good'. But later, if Michigan has Columbia as neighbor, we would have Michigan = 'good' and Columbia = 'good' — conflict. The algorithm detects `labels[neighbor] == current_label` and returns `None`.

Pseudocode

```
function ASSIGN_GOOD_AND_EVIL(graph)
    labels ← empty dictionary
    for each node in graph do                    ▷ Handle disconnected components
        if node not in labels then
            queue ← [node]
            labels[node] ← 'good'
            while queue is not empty do
                current ← queue.dequeue()
                next_label ← opposite of labels[current]
                for each neighbor of current do
                    if neighbor not in labels then
                        labels[neighbor] ← next_label
                        queue.enqueue(neighbor)
                    else if labels[neighbor] = labels[current] then
                        return None                  ▷ Odd cycle; not bipartite
                    end if
                end for
            end while
        end if
    end for
    return labels
end function
```

Python Implementation

```python
from collections import deque

def assign_good_and_evil(graph):
    labels = {}  # Maps each node to 'good' or 'evil'
    for node in graph:
        if node not in labels:
            queue = deque([node])
            labels[node] = 'good'  # Start BFS; assign first node
            while queue:
                current = queue.popleft()
                current_label = labels[current]
                next_label = 'evil' if current_label == 'good' else 'good'
                for neighbor in graph[current]:
                    if neighbor not in labels:
                        labels[neighbor] = next_label  # Alternate labels
```

```
                        queue.append(neighbor)
                    elif labels[neighbor] == current_label:
                        return None  # Conflict: odd cycle detected
    return labels

```

Time Complexity: $O(V + E)$. Each node and edge is processed at most once. **Space Complexity:** $O(V)$ for the queue and labels.

Problem 4: Network Latency Analysis

Collaborators: None.

Part 1: Unit Latency in a Tree

Task: Design an efficient algorithm to compute the maximum communication delay between any pair of data centers, assuming unit latency on each link.

Answer: The maximum delay is the **diameter** of the tree. We use the **2-Pass BFS** algorithm:

1. Pick an arbitrary node u.
2. Run BFS from u; let v be a node at maximum distance from u.
3. Run BFS from v; let w be a node at maximum distance from v.
4. The distance from v to w is the diameter (maximum delay).

Correctness: In a tree, the farthest node from any node u is always an endpoint of some diameter. Thus v is an endpoint, and the farthest node from v is the other endpoint.

Formal Proof (by Contradiction): Proof: Let T be a tree with diameter D, and let a, b be endpoints of some diameter (so $d(a, b) = D$). Let u be an arbitrary node and v a node at maximum distance from u, i.e., $d(u, v) = \max_w d(u, w)$. Assume for contradiction that v is not an endpoint of any diameter, so $v \neq a$ and $v \neq b$.

Let c be the unique node on the path a–b that is closest to u (the "projection" of u onto the diameter). In a tree, the path from u to any node on the diameter goes through c. Thus:

$$d(u, a) = d(u, c) + d(c, a), \quad d(u, b) = d(u, c) + d(c, b), \quad d(u, v) = d(u, c) + d(c, v).$$

Since v is farthest from u, we have $d(u, v) \geq d(u, a)$ and $d(u, v) \geq d(u, b)$. Hence $d(c, v) \geq d(c, a)$ and $d(c, v) \geq d(c, b)$.

The path from c to v lies entirely within the tree. If v is on the path from c to a, then $d(c, v) \leq d(c, a)$; combined with $d(c, v) \geq d(c, a)$, we get $d(c, v) = d(c, a)$, so $v = a$. Similarly, if v is on the path from c to b, then $v = b$. Thus v must equal a or b, contradicting our assumption. Therefore v is an endpoint of a diameter.

Time Complexity: $O(V + E)$. Each BFS is $O(V + E)$. For a tree, $E = V - 1$, so $O(V)$.

Part 2: Arbitrary Nonnegative Latencies

Task: Does the algorithm still work if links have arbitrary nonnegative latencies?

Answer: Yes. We replace BFS with a **weighted** shortest-path algorithm (e.g., Dijkstra's algorithm) to find the farthest node by total latency. In a tree, there is exactly one path between any two nodes, so the farthest node from u by weighted distance is still an endpoint of the diameter (longest weighted path). The second pass from that endpoint yields the correct maximum delay.

Time Complexity: $O(V + E \log V)$ with a binary heap for Dijkstra, or $O(V^2)$ with an array-based implementation. For a tree, $E = V - 1$, so $O(V \log V)$ or $O(V^2)$.

Part 3: Graphs with Cycles

(a) Counterexample for the 2-Pass Algorithm

Task: Provide an example of a connected unweighted graph with cycles where the 2-pass algorithm fails.

Counterexample 1 (Primary: Shortcuts Break 2-Pass Logic): Let $V = \{A, B, C, D, E, X\}$. Edges: path A–B–C–D–E (length 4), plus X connected to B and D. Node X creates "shortcuts" that directly break the 2-pass BFS logic without changing how the algorithm traverses.

- **True diameter:** 4 (path $A \to B \to C \to D \to E$).
- **Pass 1 from X:** BFS distances: $X(0)$, $B(1)$, $D(1)$, $A(2)$, $C(2)$, $E(2)$. Farthest distance = 2. Suppose we pick C.
- **Pass 2 from C:** BFS distances: $C(0)$, $B(1)$, $D(1)$, $A(2)$, $X(2)$, $E(2)$. Farthest distance = 2.
- **Result:** Algorithm returns 2, but the correct answer is 4.

This counterexample is strong because it directly violates the 2-pass invariant: the farthest node from an arbitrary start (X) is *not* an endpoint of the diameter. The shortcuts through X make A, C, and E all appear at distance 2 from X, so the algorithm never reaches the true diameter endpoints.

Counterexample 2 (Alternative: 4-Cycle with DFS Traversal): Consider a 4-cycle: $V = \{A, B, C, D\}$ with edges (A, B), (B, C), (C, D), (D, A). The diameter is 2. The 2-pass *BFS* algorithm correctly returns 2. However, if one mistakenly uses **DFS** (depth-first search) instead of BFS to find the "farthest" node, the traversal order can yield a wrong result: DFS explores depth-first and does not guarantee shortest-path distances, so the node deemed "farthest" by DFS tree-traversal may not be a diameter endpoint. This alternative illustrates that the algorithm crucially relies on BFS; swapping to DFS breaks correctness. (The primary counterexample above is stronger because it fails even when BFS is used correctly.)

(b) Modified Algorithm for General Unweighted Connected Graphs

Task: Modify the algorithm to correctly compute the worst-case communication delay in a general unweighted connected graph.

Answer: With cycles present, the 2-pass method fails because the farthest node from a given start may no longer lie on a longest path. We therefore compute the maximum over all pairwise shortest-path lengths. For unweighted graphs, BFS from each vertex yields shortest-path distances; the worst-case delay is the maximum of these values over all pairs.

Algorithm: For each node v, run BFS from v to obtain $\text{dist}(v, u)$ for all u. The **eccentricity** of v is $\max_u \text{dist}(v, u)$. The **diameter** (worst-case delay) is $\max_v$ (eccentricity of v). The distance map from each BFS naturally avoids revisiting nodes, so cycle edges do not distort the computed distances.

Correctness: BFS processes nodes in order of increasing distance from the source. Each node is first reached along a shortest path, so the stored distances are correct even when the graph has cycles. Taking the maximum over all BFS runs gives the longest shortest path between any two nodes.

Pseudocode

```
function BFS_DISTANCES(graph, start)
    dist[start] ← 0
    queue ← [start]
    while queue is not empty do
        u ← queue.dequeue()
        for each neighbor v of u do
            if v not in dist then
                dist[v] ← dist[u] + 1
                queue.enqueue(v)
            end if
        end for
    end while
    return dist
end function

function WORST_CASE_DELAY(graph)
    global_max ← 0
    for each node in graph do
        distances ← bfs_distances(graph, node)
        max_from_node ← max(distances.values())
        global_max ← max(global_max, max_from_node)
    end for
    return global_max
end function
```

Python Implementation

```python
from collections import deque

def bfs_distances(graph, start):
    #BFS from start; returns dict of shortest distances to each node.
    dist = {start: 0}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist

```

```python
def worst_case_delay(graph):
    #Max shortest-path length over all pairs (graph diameter).
    nodes = list(graph.keys())
    global_max = 0
    for node in nodes:
        distances = bfs_distances(graph, node)
        if distances:
            current_max = max(distances.values())
            global_max = max(global_max, current_max)
    return global_max

```

Time Complexity: A single BFS runs in $O(V + E)$. We perform V such runs, for a total of $O(V(V + E))$ — $O(V^2)$ when $E = O(V)$ and $O(V^3)$ when $E = O(V^2)$. The problem notes that faster exact algorithms are unlikely under assumptions such as SETH.

Edge Cases: Single node $\Rightarrow$ return 0. Disconnected graph: the problem assumes the graph is connected, so all nodes are reachable. If needed for disconnected graphs, we would run BFS only within the connected component of each start.